

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I _____ R.S.Adithiya/192512053 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R.S.Adithiya

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents

- Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- vi. D1 cannot work Night shift
- vii. D2 must work before D3 (Morning < Afternoon < Night)
- viii. Only one doctor per shift
- ix. D3 cannot work Morning
- x. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

ANSWER:

BACKTRACKING SEARCH FOR DOCTOR-SHIFT ASSIGNMENT

1. PROBLEM STATEMENT

A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that all the given constraints are satisfied.

Constraints:

- 1. D1 cannot work Night shift.
- 2. D2 must work before D3 (Morning < Afternoon < Night).
- 3. Only one doctor can be assigned to each shift.
- 4. D3 cannot work Morning shift.
- 5. All doctors must be assigned exactly one shift.

The objective is to apply Backtracking Search to find a valid assignment by showing all intermediate steps and constraint checks.

2. PROBLEM FORMULATION

This problem can be represented as a Constraint Satisfaction Problem (CSP).

Variables:

The variables are the three doctors:

D1, D2, D3

Domains:

Each doctor can be assigned one of the following shifts:

{Morning, Afternoon, Night}

Therefore:

$D1 \in \{\text{Morning, Afternoon, Night}\}$

$D2 \in \{\text{Morning, Afternoon, Night}\}$

$D3 \in \{\text{Morning, Afternoon, Night}\}$

Constraints:

C1: $D1 \neq \text{Night}$

C2: D2 must work before D3

C3: No two doctors can be assigned the same shift

C4: $D3 \neq \text{Morning}$

C5: Every doctor must be assigned exactly one shift

The objective is to find an assignment that satisfies all the above constraints.

3. BACKTRACKING SEARCH

Backtracking Search is a systematic search technique used to solve Constraint Satisfaction Problems. It assigns values to variables one by one and checks whether the constraints are satisfied.

If an assignment satisfies all current constraints, the algorithm continues to the next variable.

If an assignment violates any constraint, the algorithm rejects that assignment and goes back to the previous variable to try another possible value.

The general process is:

SELECT VARIABLE → ASSIGN SHIFT → CHECK CONSTRAINTS → CONTINUE OR BACKTRACK

For this problem, the variable assignment order is:

D1 → D2 → D3

The shifts are considered in the order:

Morning → Afternoon → Night

4. STEP-BY-STEP BACKTRACKING SEARCH

STEP 1: ASSIGN D1 = MORNING

Current assignment:

D1 = Morning

D2 = Not Assigned

D3 = Not Assigned

Constraint Checks:

C1: D1 cannot work Night.

D1 = Morning, so the constraint is satisfied.

C3: Only one doctor per shift.

Only D1 is assigned to Morning, so the constraint is satisfied.

Therefore:

D1 = Morning is a valid assignment.

Continue to D2.

STEP 2: TRY D2 = MORNING

Current assignment:

D1 = Morning

D2 = Morning

D3 = Not Assigned

Constraint Check:

C3: Only one doctor per shift.

D1 and D2 are both assigned to the Morning shift.

Therefore, the constraint is violated.

Result:

D2 = Morning is rejected.

Action:

BACKTRACK and try the next possible shift for D2.

STEP 3: TRY D2 = AFTERNOON

Current assignment:

D1 = Morning

D2 = Afternoon

D3 = Not Assigned

Constraint Checks:

C3: Only one doctor per shift.

D1 = Morning and D2 = Afternoon.

Both doctors have different shifts.

Therefore, the constraint is satisfied.

Now, D2 must work before D3.

Since D2 = Afternoon, D3 must work after Afternoon.

Therefore:

D3 = Night

Continue to D3.

STEP 4: ASSIGN D3 = NIGHT

Current assignment:

D1 = Morning

D2 = Afternoon

D3 = Night

Now check all constraints.

C1: D1 cannot work Night.

D1 = Morning.

Constraint satisfied.

C2: D2 must work before D3.

D2 = Afternoon and D3 = Night.

Afternoon < Night.

Constraint satisfied.

C3: Only one doctor per shift.

D1 = Morning

D2 = Afternoon

D3 = Night

All three doctors have different shifts.

Constraint satisfied.

C4: D3 cannot work Morning.

D3 = Night.

Constraint satisfied.

C5: All doctors must be assigned exactly one shift.

D1, D2, and D3 are all assigned exactly one shift.

Constraint satisfied.

Since all constraints are satisfied, the algorithm reaches the goal state.

5. SEARCH TREE

The Backtracking Search process can be represented as:

START

↓

D1 = Morning

↓

Try D2 = Morning

↓

Constraint C3 Violated

↓

BACKTRACK

↓

Try D2 = Afternoon

↓

D3 = Night

↓

Check All Constraints

↓

C1 Satisfied

C2 Satisfied

C3 Satisfied

C4 Satisfied

C5 Satisfied

↓

VALID SOLUTION FOUND

6. INTERMEDIATE STEPS AND CONSTRAINT CHECKS

Step	Assignment	Constraint Check	Result	Action
1	D1 = Morning	D1 cannot be Night	Satisfied	Continue
2	D2 = Morning	D1 and D2 cannot share the same shift	Violated	Backtrack
3	D2 = Afternoon	D1 and D2 have different shifts	Satisfied	Continue
4	D3 = Night	D2 must work before D3	Satisfied	Continue
5	D3 = Night	D3 cannot work Morning	Satisfied	Continue
6	D1 = Morning, D2 = Afternoon, D3 = Night	Only one doctor per shift	Satisfied	Continue
7	All doctors assigned	All constraints satisfied	Satisfied	Goal Reached

7. FINAL VALID SCHEDULE

The final valid assignment is:

DOCTOR ASSIGNED SHIFT

D1 MORNING

D2 AFTERNOON

D3 NIGHT

Schedule Based on Shifts:

MORNING → D1

AFTERNOON → D2

NIGHT → D3

Therefore, the final valid solution is:

D1 = Morning

D2 = Afternoon

D3 = Night

8. VERIFICATION OF THE FINAL SOLUTION

The final assignment is:

D1 = Morning

D2 = Afternoon

D3 = Night

Constraint 1: D1 cannot work Night shift.

D1 is assigned to Morning.

Therefore, Constraint 1 is satisfied.

Constraint 2: D2 must work before D3.

D2 is assigned to Afternoon and D3 is assigned to Night.

Afternoon < Night.

Therefore, Constraint 2 is satisfied.

Constraint 3: Only one doctor per shift.

Morning → D1

Afternoon → D2

Night → D3

Each shift has exactly one doctor.

Therefore, Constraint 3 is satisfied.

Constraint 4: D3 cannot work Morning shift.

D3 is assigned to Night.

Therefore, Constraint 4 is satisfied.

Constraint 5: All doctors must be assigned exactly one shift.

D1 → Morning

D2 → Afternoon

D3 → Night

All three doctors are assigned exactly one shift.

Therefore, Constraint 5 is satisfied.

CONCLUSION

The Backtracking Search algorithm successfully finds a valid schedule by assigning shifts to doctors one at a time and checking the constraints after every assignment.

Initially, the assignment D1 = Morning and D2 = Morning is rejected because two doctors cannot be assigned to the same shift. The algorithm then backtracks and tries D2 = Afternoon. Since D2 must work before D3, D3 is assigned to the Night shift.

The final assignment is:

MORNING → D1

AFTERNOON → D2

NIGHT → D3

Thus, the final solution is:

{D1 = Morning, D2 = Afternoon, D3 = Night}

All five constraints are satisfied. Therefore, the hospital has obtained a valid and conflict-free doctor-shift schedule using the Backtracking Search algorithm.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- v. Movement allowed: Up, Down, Left, Right
- vi. Cost of each move = 1
- vii. Cannot pass through obstacles
- viii. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

ANSWER:

BFS SEARCH ALGORITHM FOR ROBOT GRID NAVIGATION

1. PROBLEM STATEMENT

A robot operates in a 5×5 grid environment. The robot must move from a Start (S) position to a Goal (G) position. Some cells in the grid contain obstacles that block the robot's movement.

The objective is to use the Breadth-First Search (BFS) algorithm to find the optimal path from Start (S) to Goal (G).

Constraints:

1. Movement is allowed only in four directions:
 - Up
 - Down
 - Left
 - Right
2. Cost of each move = 1.
3. The robot cannot move through obstacles.
4. Manhattan Distance heuristic is given as:

$$\text{Manhattan Distance} = |x_1 - x_2| + |y_1 - y_2|$$

5. BFS Search Algorithm must be used to find the optimal path.

2. SAMPLE 5×5 GRID

For this solution, consider the following 5×5 grid.

S represents the Start position.

G represents the Goal position.

X represents an Obstacle.

. represents a Free Cell.

The grid is:

C1 C2 C3 C4 C5

R1 S . . X .

R2 X X . X .

R3

R4 . X X X .

R5 G

Therefore:

Start (S) = (1,1)

Goal (G) = (5,5)

Obstacles = (1,4), (2,1), (2,2), (2,4), (4,2), (4,3), (4,4)

The robot can move only through free cells.

3. PROBLEM FORMULATION

This problem can be represented as a state-space search problem.

Initial State

The initial state is the Start position:

S = (1,1)

Goal State

The goal state is:

G = (5,5)

State

Each free cell in the grid represents a state.

For example:

(1,1), (1,2), (1,3), (2,3), etc.

Actions

The robot can perform four actions:

- 1. Move Up**
- 2. Move Down**
- 3. Move Left**
- 4. Move Right**

Transition Model

When the robot performs a valid movement action, it moves from the current cell to an adjacent free cell.

The robot cannot move:

- Outside the grid.**
- Into an obstacle.**
- Into an already explored state.**

Path Cost

Each movement has a cost of 1.

Therefore:

Path Cost = Number of Moves

Goal Test

The search is successful when the robot reaches:

$G = (5,5)$

4. MANHATTAN DISTANCE HEURISTIC

The Manhattan Distance between two cells is calculated as:

Manhattan Distance = $|x1 - x2| + |y1 - y2|$

For example, for the Start position $S = (1,1)$ and Goal position $G = (5,5)$:

Manhattan Distance = $|1 - 5| + |1 - 5|$

$$= 4 + 4$$

$$= 8$$

Therefore, the Manhattan Distance from S to G is 8.

However, since the question specifically asks for BFS, the Manhattan Distance is not used to decide which node is expanded first. BFS expands nodes level by level.

The Manhattan Distance can be used to understand the minimum possible number of movements in an obstacle-free grid.

5. BREADTH-FIRST SEARCH (BFS)

Breadth-First Search explores the search space level by level.

The algorithm uses a FIFO Queue.

The node that enters the queue first is expanded first.

The search process is:

START → Explore all nodes at depth 1 → Explore all nodes at depth 2 → Continue until GOAL is reached.

Since every movement has the same cost of 1, BFS guarantees an optimal path.

Therefore:

Optimal Path = Shortest Path

Optimal Cost = Minimum Number of Moves

6. BFS SEARCH PROCESS

The search begins at:

S = (1,1)

Initial Queue:

[(1,1)]

Visited:

{(1,1)}

STEP 1: EXPAND (1,1)

Current Node:

(1,1)

Possible movements:

UP → Outside Grid → Invalid

DOWN → (2,1) → Obstacle → Invalid

LEFT → Outside Grid → Invalid

RIGHT → (1,2) → Valid

Therefore, only (1,2) is added.

Queue Update:

Before Expansion:

[(1,1)]

After Expansion:

[(1,2)]

Visited:

{(1,1), (1,2)}

STEP 2: EXPAND (1,2)

Current Node:

(1,2)

Possible movements:

UP → Outside Grid → Invalid

DOWN → (2,2) → Obstacle → Invalid

LEFT → (1,1) → Already Visited

RIGHT $\rightarrow$ (1,3) $\rightarrow$ Valid

Queue Update:

Before Expansion:

[(1,2)]

After Expansion:

[(1,3)]

Visited:

{(1,1), (1,2), (1,3)}

STEP 3: EXPAND (1,3)

Current Node:

(1,3)

Possible movements:

UP $\rightarrow$ Outside Grid $\rightarrow$ Invalid

DOWN $\rightarrow$ (2,3) $\rightarrow$ Valid

LEFT $\rightarrow$ (1,2) $\rightarrow$ Already Visited

RIGHT $\rightarrow$ (1,4) $\rightarrow$ Obstacle

Add:

(2,3)

Queue Update:

Before Expansion:

[(1,3)]

After Expansion:

[(2,3)]

Visited:

{(1,1), (1,2), (1,3), (2,3)}

STEP 4: EXPAND (2,3)

Current Node:

(2,3)

Possible movements:

UP → (1,3) → Already Visited

DOWN → (3,3) → Valid

LEFT → (2,2) → Obstacle

RIGHT → (2,4) → Obstacle

Add:

(3,3)

Queue Update:

Before Expansion:

[(2,3)]

After Expansion:

[(3,3)]

Visited:

{(1,1), (1,2), (1,3), (2,3), (3,3)}

STEP 5: EXPAND (3,3)

Current Node:

(3,3)

Possible movements:

UP → (2,3) → Already Visited

DOWN $\rightarrow (4,3) \rightarrow$ Obstacle

LEFT $\rightarrow (3,2) \rightarrow$ Valid

RIGHT $\rightarrow (3,4) \rightarrow$ Valid

Add:

(3,2), (3,4)

Queue Update:

Before Expansion:

[(3,3)]

After Expansion:

[(3,2), (3,4)]

Visited:

{(1,1), (1,2), (1,3), (2,3), (3,3), (3,2), (3,4)}

STEP 6: EXPAND (3,2)

Current Node:

(3,2)

Possible movements:

UP $\rightarrow (2,2) \rightarrow$ Obstacle

DOWN $\rightarrow (4,2) \rightarrow$ Obstacle

LEFT $\rightarrow (3,1) \rightarrow$ Valid

RIGHT $\rightarrow (3,3) \rightarrow$ Already Visited

Add:

(3,1)

Queue Update:

Before Expansion:

[(3,2), (3,4)]

After Expansion:

[(3,4), (3,1)]

Visited:

{(1,1), (1,2), (1,3), (2,3), (3,3), (3,2), (3,4), (3,1)}

STEP 7: EXPAND (3,4)

Current Node:

(3,4)

Possible movements:

UP → (2,4) → Obstacle

DOWN → (4,4) → Obstacle

LEFT → (3,3) → Already Visited

RIGHT → (3,5) → Valid

Add:

(3,5)

Queue Update:

Before Expansion:

[(3,4), (3,1)]

After Expansion:

[(3,1), (3,5)]

Visited:

{(1,1), (1,2), (1,3), (2,3), (3,3), (3,2), (3,4), (3,1), (3,5)}

STEP 8: EXPAND (3,1)

Current Node:

(3,1)

Possible movements:

UP → (2,1) → Obstacle

DOWN → (4,1) → Valid

LEFT → Outside Grid → Invalid

RIGHT → (3,2) → Already Visited

Add:

(4,1)

Queue Update:

Before Expansion:

[(3,1), (3,5)]

After Expansion:

[(3,5), (4,1)]

Visited:

{(1,1), (1,2), (1,3), (2,3), (3,3), (3,2), (3,4), (3,1), (3,5), (4,1)}

STEP 9: EXPAND (3,5)

Current Node:

(3,5)

Possible movements:

UP → (2,5) → Valid

DOWN → (4,5) → Valid

LEFT → (3,4) → Already Visited

RIGHT → Outside Grid → Invalid

Add:

(2,5), (4,5)

Queue Update:

Before Expansion:

[(3,5), (4,1)]

After Expansion:

[(4,1), (2,5), (4,5)]

STEP 10: EXPAND (4,1)

Current Node:

(4,1)

Possible movements:

UP → (3,1) → Already Visited

DOWN → (5,1) → Valid

LEFT → Outside Grid → Invalid

RIGHT → (4,2) → Obstacle

Add:

(5,1)

Queue Update:

Before Expansion:

[(4,1), (2,5), (4,5)]

After Expansion:

[(2,5), (4,5), (5,1)]

STEP 11: EXPAND (2,5)

Current Node:

(2,5)

Possible movements:

UP → (1,5) → Valid

DOWN → (3,5) → Already Visited

LEFT → (2,4) → Obstacle

RIGHT → Outside Grid → Invalid

Add:

(1,5)

Queue Update:

Before Expansion:

[(2,5), (4,5), (5,1)]

After Expansion:

[(4,5), (5,1), (1,5)]

STEP 12: EXPAND (4,5)

Current Node:

(4,5)

Possible movements:

UP → (3,5) → Already Visited

DOWN → (5,5) → GOAL

LEFT → (4,4) → Obstacle

RIGHT → Outside Grid → Invalid

The Goal (5,5) is found.

Therefore, BFS terminates.

7. BFS NODE EXPANSION TABLE

Step	Expanded Node	New Valid Nodes	Queue After Expansion
1	(1,1)	(1,2)	(1,2)
2	(1,2)	(1,3)	(1,3)
3	(1,3)	(2,3)	(2,3)
4	(2,3)	(3,3)	(3,3)
5	(3,3)	(3,2), (3,4)	(3,2), (3,4)
6	(3,2)	(3,1)	(3,4), (3,1)
7	(3,4)	(3,5)	(3,1), (3,5)
8	(3,1)	(4,1)	(3,5), (4,1)
9	(3,5)	(2,5), (4,5)	(4,1), (2,5), (4,5)
10	(4,1)	(5,1)	(2,5), (4,5), (5,1)
11	(2,5)	(1,5)	(4,5), (5,1), (1,5)
12	(4,5)	(5,5) Goal	GOAL FOUND

8. OPTIMAL PATH

Tracing the parent nodes from the Goal back to the Start:

G = (5,5)

↓

(4,5)

↓

(3,5)

↓

(3,4)

↓

(3,3)

↓

(2,3)

↓

(1,3)

↓

(1,2)

↓

S = (1,1)

Therefore, the optimal path from Start to Goal is:

(1,1) → (1,2) → (1,3) → (2,3) → (3,3) → (3,4) → (3,5) → (4,5) → (5,5)

In terms of movements:

RIGHT → RIGHT → DOWN → DOWN → RIGHT → RIGHT → DOWN → DOWN

9. PATH VISUALIZATION

The optimal path is represented by *.

S * * X .

X X * X .

.. * * *

. X X X *

.... G

Therefore:

S → → ↓ ↓ → → ↓ ↓ → G

10. PATH COST CALCULATION

Each movement has a cost of 1.

The optimal path contains 8 movements:

1. $(1,1) \rightarrow (1,2)$
2. $(1,2) \rightarrow (1,3)$
3. $(1,3) \rightarrow (2,3)$
4. $(2,3) \rightarrow (3,3)$
5. $(3,3) \rightarrow (3,4)$
6. $(3,4) \rightarrow (3,5)$
7. $(3,5) \rightarrow (4,5)$
8. $(4,5) \rightarrow (5,5)$

Therefore:

$$\text{Total Path Cost} = 8 \times 1$$

$$\text{Total Path Cost} = 8$$

Hence:

$$\text{OPTIMAL PATH COST} = 8$$

11. WHY BFS GIVES THE OPTIMAL PATH

BFS explores nodes level by level.

Since every movement has the same cost of 1, the first time BFS reaches the Goal, it has found the path with the minimum number of moves.

Therefore:

BFS is COMPLETE.

BFS is OPTIMAL when all step costs are equal.

In this problem:

$$\text{Cost of each move} = 1$$

Therefore, the path found by BFS is the optimal path.

The Manhattan Distance from Start to Goal is also:

$$|1 - 5| + |1 - 5| = 8$$

The obtained path cost is also 8.

Thus, the path achieves the minimum possible cost.

CONCLUSION

The 5×5 robot grid navigation problem can be solved using the Breadth-First Search (BFS) algorithm. The robot starts at $S = (1,1)$ and must reach $G = (5,5)$ while avoiding obstacles. The robot can move only Up, Down, Left, or Right, and each movement has a cost of 1.

BFS explores the nodes level by level using a FIFO queue. At each step, invalid moves caused by obstacles, boundaries, or previously visited cells are rejected. The queue is updated with valid neighboring nodes.

For the given sample grid, BFS finds the following optimal path:

$(1,1) \rightarrow (1,2) \rightarrow (1,3) \rightarrow (2,3) \rightarrow (3,3) \rightarrow (3,4) \rightarrow (3,5) \rightarrow (4,5) \rightarrow (5,5)$

The corresponding movement sequence is:

RIGHT $\rightarrow$ RIGHT $\rightarrow$ DOWN $\rightarrow$ DOWN $\rightarrow$ RIGHT $\rightarrow$ RIGHT $\rightarrow$ DOWN $\rightarrow$ DOWN

The total number of movements is 8, and since each movement has a cost of 1:

OPTIMAL PATH COST = 8

Therefore, BFS successfully finds the shortest and optimal path from the Start position S to the Goal position G while avoiding all obstacles.

1. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

ANSWER:

AUTONOMOUS RESCUE ROBOT USING ONLINE SEARCH AND UNIFORM COST SEARCH

1. INTRODUCTION

An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The building is represented as a grid of rooms. The robot starts from a position called S (Start) and must reach a survivor located at G (Goal).

The environment is challenging because some paths may be blocked, the robot has limited sensing capability, and it does not initially have complete information about the building. The robot can observe only adjacent cells and must dynamically update its knowledge as it moves.

The robot can move in four directions:

- 1. Up**
- 2. Down**
- 3. Left**
- 4. Right**

Every normal movement has a cost of 1. However, entering a risky zone adds an additional cost of 2. Therefore, entering a risky zone has a total movement cost of:

Normal movement cost + Risk cost

$$= 1 + 2$$

$$= 3$$

The main objective is to reach the survivor while minimizing the total cost and ensuring safe navigation.

This problem can be solved using a combination of an Online Search Agent and Uniform Cost Search (UCS).

2. TASK (A): ANALYSIS OF THE GIVEN CONSTRAINTS

The constraints have a direct effect on the robot's decision-making process.

Constraint 1: Movement is allowed only Up, Down, Left, and Right

The robot can move only to adjacent cells in four directions.

The robot cannot move diagonally.

The possible actions are:

UP → Move to the cell above

DOWN → Move to the cell below

LEFT → Move to the cell on the left

RIGHT → Move to the cell on the right

Effect on Decision-Making:

The robot must check the four neighboring cells before selecting its next movement. It must choose a valid direction that leads toward the survivor while avoiding obstacles and risky areas whenever possible.

Constraint 2: Some Cells are Blocked

Some cells in the building contain obstacles or blocked paths.

The robot cannot enter these cells.

Effect on Decision-Making:

The robot must identify blocked cells using its sensors. If a path is blocked, the robot must reject that path and search for an alternative route.

Since the robot does not initially know all blocked paths, it may discover new obstacles while moving.

Therefore, the robot must update its internal knowledge dynamically.

Constraint 3: Limited Sensing Capability

The robot can observe only adjacent cells.

It cannot see the entire building or know the complete map in advance.

Effect on Decision-Making:

The environment is partially observable.

The robot must:

- 1. Observe nearby cells.**
- 2. Update its internal map.**
- 3. Identify possible movements.**
- 4. Select the best action.**
- 5. Move to the selected cell.**
- 6. Sense the new surroundings.**
- 7. Update its knowledge again.**

Therefore, the robot cannot plan the complete route from the beginning. It must make decisions based on the information currently available.

Constraint 4: Movement Cost

Every normal movement has a cost of 1.

For example:

Normal Cell:

Movement Cost = 1

The robot should minimize the total number of movements whenever possible.

Constraint 5: Risky Zones

Entering a risky zone has an additional cost of 2.

Therefore:

Normal Movement Cost = 1

Additional Risk Cost = 2

Total Risky Zone Cost = $1 + 2 = 3$

Effect on Decision-Making:

The robot should avoid risky zones if a safer route with a reasonable cost is available.

For example:

Path A:

5 normal movements

Total Cost = $5 \times 1 = 5$

Path B:

3 normal movements + 1 risky zone

Total Cost = $3 + 3 = 6$

Although Path B is shorter in terms of distance, Path A is cheaper and safer.

Therefore, the robot should choose Path A.

This demonstrates why the robot must consider both distance and risk.

Constraint 6: Minimize Total Cost

The robot must find a route that minimizes the total movement and risk cost.

The total cost can be calculated as:

TOTAL COST = NORMAL MOVEMENT COST + RISK COST

For every normal cell:

Cost = 1

For every risky cell:

Cost = $1 + 2 = 3$

The robot should select the path with the lowest cumulative cost.

Constraint 7: Dynamic Knowledge Updating

The robot must update its knowledge as it explores the building.

Initially:

Knowledge = Limited

After sensing:

Knowledge = Updated

The robot continuously follows:

SENSE → UPDATE MAP → PLAN → MOVE → SENSE AGAIN

If the robot discovers a blocked path, it must revise its plan.

If the robot discovers a risky zone, it must consider whether another safer path is better.

Therefore, the robot requires an online search strategy.

3. TASK (B): SOLUTION APPROACH

Selected Approach: Online Search Agent with Uniform Cost Search

The most suitable solution is a combination of:

ONLINE SEARCH AGENT + UNIFORM COST SEARCH (UCS)

The Online Search Agent allows the robot to make decisions using incomplete information and update its knowledge dynamically.

Uniform Cost Search helps the robot choose the path with the minimum cumulative cost.

The combined approach is suitable because:

- The robot does not know the complete environment.
- Obstacles may be discovered during exploration.
- Risky zones have additional costs.
- The robot needs to minimize total cost.
- The robot must continuously update its knowledge.

4. WORKING OF THE PROPOSED APPROACH

The robot follows the following cycle:

SENSE → UPDATE → SEARCH → SELECT → MOVE → REPEAT

Step 1: Start at S

The robot begins at the starting position S.

The initial path cost is:

$\text{Cost}(S) = 0$

Step 2: Sense the Environment

The robot observes the adjacent cells.

It checks:

- Is the cell inside the grid?
- Is the cell blocked?
- Is the cell risky?
- Is the cell safe?
- Has the cell already been visited?

Step 3: Update the Internal Map

The robot stores the information collected from its sensors.

For example:

UNKNOWN → FREE

UNKNOWN → BLOCKED

UNKNOWN → RISKY

The robot gradually builds an internal map of the building.

Step 4: Calculate Movement Costs

For every available neighboring cell:

Normal cell:

$\text{Cost} = 1$

Risky cell:

$\text{Cost} = 1 + 2 = 3$

Blocked cell:

$\text{Cost} = \infty$ or Not Allowed

The robot calculates the cumulative cost of reaching each possible state.

Step 5: Select the Minimum-Cost Path

The robot uses Uniform Cost Search to select the state with the lowest cumulative cost.

The priority value is:

$g(n) = \text{Total Cost from S to Current Node}$

The node with the smallest $g(n)$ is selected first.

Step 6: Move to the Selected Cell

The robot moves to the selected cell.

After moving, it senses the new surroundings again.

Step 7: Update the Knowledge

The robot updates its internal map based on newly discovered information.

If a new obstacle is discovered, the robot removes that route from consideration.

If a risky zone is discovered, the robot assigns the appropriate additional cost.

Step 8: Re-plan if Necessary

If the current path becomes unavailable or unsafe, the robot performs a new search using the updated information.

The robot does not blindly follow the original plan.

Instead, it dynamically changes its route.

Step 9: Reach the Survivor

The process continues until the robot reaches G.

When:

Current Position = G

the search terminates.

The robot has successfully located the survivor.

5. ONLINE SEARCH PROCESS

The overall process can be represented as:

START AT S

↓

SENSE ADJACENT CELLS

↓

UPDATE INTERNAL MAP

↓

IDENTIFY VALID MOVEMENTS

↓

CALCULATE MOVEMENT COSTS

↓

SELECT LOWEST-COST OPTION

↓

MOVE TO NEXT CELL

↓

SENSE AGAIN

↓

UPDATE KNOWLEDGE

↓

IS PATH SAFE AND AVAILABLE?

/

YES

|

↓

\

NO

|

↓

CONTINUE RE-PLAN

||
↓|

REACH G? ←—————|

|

YES

|

↓

SURVIVOR FOUND

|

END

6. UNIFORM COST SEARCH PSEUDOCODE

The following pseudocode represents the proposed Online Search Agent using Uniform Cost Search.

BEGIN

Set **current_position** = S

Set **total_cost** = 0

Initialize **internal_map**

WHILE **current_position** != G **DO**

SENSE adjacent cells

UPDATE **internal_map**

FOR each adjacent cell **DO**

IF cell is blocked **THEN**

 Mark cell as **BLOCKED**

 Do not add cell to search queue

ELSE

IF cell is risky **THEN**

MovementCost = 1 + 2

ELSE

MovementCost = 1

END IF

 Calculate cumulative cost

```

        Add cell to priority queue

    END IF

END FOR

Select the cell with minimum cumulative cost

IF selected cell is safe and reachable THEN

    MOVE to selected cell

    total_cost = updated cumulative cost

ELSE

    REPLAN using updated internal map

END IF

UPDATE internal_map

END WHILE

IF current_position = G THEN

    DISPLAY "Survivor Found"

    DISPLAY total_cost

END IF
END

```

7. TASK (C): APPLICATION OF AI CONCEPTS

7.1 INTELLIGENT AGENT

An Intelligent Agent is a system that perceives its environment through sensors and takes actions to achieve a goal.

The rescue robot acts as an Intelligent Agent.

The robot follows:

PERCEPTION → DECISION → ACTION

Perception:

The robot uses sensors to observe:

- **Adjacent cells**
- **Obstacles**
- **Risky zones**
- **Free paths**
- **Survivor location when detected**

Decision:

The robot analyzes the available information and selects the safest and least-cost action.

Action:

The robot performs one of the following actions:

- **Move Up**
- **Move Down**
- **Move Left**
- **Move Right**
- **Re-plan the route**

Therefore, the rescue robot is an example of an autonomous intelligent agent.

7.2 RATIONALITY

A rational agent chooses the best possible action based on the information currently available to it.

The rescue robot must make rational decisions because it operates in an uncertain disaster environment.

For example, suppose the robot has two possible paths.

Path A:

- **Shorter distance**
- **Contains a risky zone**

Path B:

- **Slightly longer**
- **Completely safe**

The robot compares the total costs.

If:

Path A Cost = 6

Path B Cost = 5

The rational decision is to select Path B.

The robot therefore considers:

- **Total path cost**
- **Safety**
- **Obstacles**
- **Risk**
- **Available information**

The robot's goal is to maximize its performance by reaching the survivor safely with minimum total cost.

Thus, the robot demonstrates rational behavior.

7.3 ENVIRONMENT TYPE

The rescue robot operates in a complex AI environment.

The environment can be characterized as follows:

1. Partially Observable

The robot cannot observe the entire building.

It can sense only adjacent cells.

Therefore:

Environment = Partially Observable

2. Stochastic

The robot may encounter unexpected obstacles or changes in the environment.

Therefore, the environment can be considered:

Environment = Stochastic

3. Sequential

Every movement affects the robot's future position and future decisions.

Therefore:

Environment = Sequential

4. Dynamic

The disaster environment may change over time.

For example:

- Paths may become blocked.
- Risky areas may become more dangerous.
- New information may become available.

Therefore:

Environment = Dynamic

5. Discrete

The building is represented as a grid of individual cells.

The robot moves from one cell to another.

Therefore:

Environment = Discrete

6. Single-Agent

The robot is primarily responsible for making decisions and reaching the survivor.

Therefore:

Environment = Single-Agent

Overall Environment Classification

The rescue robot operates in a:

Partially Observable, Stochastic, Sequential, Dynamic, Discrete, Single-Agent Environment

8. TASK (D): JUSTIFICATION OF THE CHOSEN APPROACH

The combination of an Online Search Agent and Uniform Cost Search is the most suitable approach for this problem.

Reason 1: Partial Observability

The robot cannot see the entire building.

An Online Search Agent can make decisions using the information currently available.

It does not require a complete map at the beginning.

Reason 2: Dynamic Environment

The disaster environment may change.

New obstacles may be discovered during exploration.

The Online Search Agent can update its knowledge and re-plan the route.

Reason 3: Different Movement Costs

Normal cells have a cost of 1.

Risky cells have a cost of 3.

UCS is suitable because it considers the cumulative cost of the path instead of simply choosing the shortest distance.

Reason 4: Safety Consideration

The robot should avoid unnecessary risky areas.

By assigning an additional cost to risky zones, the search algorithm naturally prefers safer routes when they have lower total cost.

Reason 5: Optimal Cost

Uniform Cost Search expands the node with the smallest cumulative cost first. Therefore, when all costs are non-negative, UCS can find the least-cost path based on the currently known map.

Reason 6: Dynamic Re-planning

If the robot discovers that the planned path is blocked, it can update its internal map and perform another search.

This makes the approach suitable for online navigation.

9. COMPARISON WITH OTHER SEARCH STRATEGIES

Search Strategy	Suitability	Reason
BFS	Less Suitable	Treats all movements equally and does not properly consider risky-zone costs
DFS	Not Suitable	May choose a long or dangerous path and does not guarantee minimum cost
Greedy Best-First Search	Less Suitable	May move toward the goal without considering total path cost or risk
A* Search	Suitable if reliable heuristic and updated map are available	Uses cost and heuristic information, but the environment is partially observable
Uniform Cost Search	Highly Suitable	Handles different path costs and finds the least-cost route
Online Search Agent	Highly Suitable	Works with incomplete information and updates knowledge dynamically
UCS + Online Search	Most Suitable	Combines minimum-cost path selection with dynamic exploration and re-planning

10. OVERALL SYSTEM ARCHITECTURE

The complete AI-based rescue robot can be represented as:

SENSORS



PERCEPTION



INTERNAL MAP / KNOWLEDGE BASE



ONLINE SEARCH



UNIFORM COST SEARCH



COST AND SAFETY EVALUATION



ACTION SELECTION



MOVE



NEW PERCEPTION

↓
UPDATE KNOWLEDGE
↓
RE-PLAN IF REQUIRED
↓
REACH SURVIVOR

11. PERFORMANCE MEASURE

The performance of the autonomous rescue robot can be evaluated using the following factors:

1. Minimum total movement cost.
2. Minimum exposure to risky zones.
3. Safe navigation.
4. Successful avoidance of obstacles.
5. Successful discovery of the survivor.
6. Minimum time required to reach the survivor.
7. Efficient use of available information.
8. Ability to dynamically re-plan when new obstacles are discovered.

A suitable performance measure is:

Maximize survivor rescue success and safety while minimizing total movement cost, risk exposure, and travel time.

CONCLUSION

The autonomous rescue robot problem is a complex Artificial Intelligence problem because the robot operates in a partially observable and dynamic environment. It cannot initially see the complete building and can observe only adjacent cells. It must avoid obstacles, consider risky zones, minimize total cost, and dynamically update its knowledge.

The most suitable solution is a combination of an Online Search Agent and Uniform Cost Search (UCS). The Online Search Agent enables the robot to explore the environment and update its internal map dynamically. Uniform Cost Search allows the robot to select paths based on their cumulative cost, where normal movements have a cost of 1 and risky zones have an additional cost of 2.

The robot follows the cycle:

SENSE → UPDATE → SEARCH → SELECT → MOVE → RE-SENSE → RE-PLAN

The rescue robot demonstrates important AI concepts such as Intelligent Agents, Rationality, Environment Classification, State-Space Search, Cost-Based Decision Making, and Online Search.

Therefore, the combination of Online Search and Uniform Cost Search is the most appropriate approach because it enables the robot to make rational decisions with incomplete information, safely navigate a changing environment, dynamically update its knowledge, and find a low-cost route to the survivor.